24 x 15 m Dual LiDAR 명령어 매뉴얼

0. 공통

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash
```

1. 네트워크 확인

```
nmcli con up lidar-network
ip -brief addr show enp12s0
ping -c 3 192.168.6.11
ping -c 3 192.168.6.12
ip -s link show enp12s0
```

2. 캘리브레이션 시작

```
CAL_SESSION="field_24x15_origin00_$(date +%Y%m%d_%H%M%S)"
printf '%s\n' "$CAL_SESSION" > /tmp/grid_calibration_session
mkdir -p "$HOME/grid_calibration/$CAL_SESSION"
```

터미널 1 — PTP

```
cd ~/drone_ws
sudo ptp4l -f ~/drone_ws/ptp_gm.conf -m
```

터미널 2 — Dual LiDAR

```
source ~/drone_ws/install/setup.bash
ros2 launch dual_ouster_driver dual_os1.launch.py
```

입력 확인

```
ros2 topic hz /lidar1/points
ros2 topic hz /lidar2/points
ros2 topic info -v /lidar1/points
ros2 topic info -v /lidar2/points
```

3. 센서 방향 확인

```
source ~/drone_ws/install/setup.bash
ros2 run dual_ouster_driver axis_probe --ros-args \
  -p input_topic:=/lidar1/points \
  -p yaw_deg:=0.0 \
  -p z_band:=1.2 \
  -p max_range:=6.0
```

```
source ~/drone_ws/install/setup.bash
ros2 run dual_ouster_driver axis_probe --ros-args \
  -p input_topic:=/lidar2/points \
  -p yaw_deg:=0.0 \
  -p z_band:=1.2 \
  -p max_range:=6.0
```

4. 지면 보정

LiDAR 1

```
source ~/drone_ws/install/setup.bash
ros2 run grid_sensor_calibration ground_plane_node --ros-args \
  -p input_topic:=/lidar1/points \
  -p sensor_label:=lidar1 \
  -p expected_height:=2.0 \
  -p num_frames:=20 \
  -p range_max:=10.0 \
  -p azimuth_min:=90.0 \
  -p azimuth_max:=-90.0
```

LiDAR 2

```
source ~/drone_ws/install/setup.bash
ros2 run grid_sensor_calibration ground_plane_node --ros-args \
  -p input_topic:=/lidar2/points \
  -p sensor_label:=lidar2 \
  -p expected_height:=2.0 \
  -p num_frames:=20 \
  -p range_max:=10.0 \
  -p azimuth_min:=90.0 \
  -p azimuth_max:=-90.0
```

5A. 자동 원통 검출

터미널 3 — LiDAR 1

```
source ~/drone_ws/install/setup.bash
ros2 run grid_sensor_calibration pointcloud_target_node --ros-args \
  -p input_topic:=/lidar1/points \
  -p output_topic:=/sensor1/target_point \
  -p frame_id:=lidar1 \
  -p 'box:=[-30.0,30.0,-30.0,30.0,-1.68,-0.28]' \
  -p range_min:=1.0 \
  -p range_max:=22.0 \
  -p azimuth_min:=90.0 \
  -p azimuth_max:=-90.0 \
  -p cluster_tolerance:=0.20 \
  -p cluster_min_points:=5 \
  -p cluster_max_extent:=2.2 \
  -p selection:=largest \
  -p target_shape:=cylinder \
  -p target_radius:=0.15 \
  -p max_fit_residual:=0.04 \
  -p voxel_size:=0.0
```

터미널 4 — LiDAR 2

```
source ~/drone_ws/install/setup.bash
ros2 run grid_sensor_calibration pointcloud_target_node --ros-args \
  -p input_topic:=/lidar2/points \
  -p output_topic:=/sensor2/target_point \
  -p frame_id:=lidar2 \
  -p 'box:=[-30.0,30.0,-30.0,30.0,-1.68,-0.28]' \
  -p range_min:=1.0 \
  -p range_max:=22.0 \
  -p azimuth_min:=90.0 \
  -p azimuth_max:=-90.0 \
  -p cluster_tolerance:=0.20 \
  -p cluster_min_points:=5 \
  -p cluster_max_extent:=2.2 \
  -p selection:=largest \
  -p target_shape:=cylinder \
  -p target_radius:=0.15 \
  -p max_fit_residual:=0.04 \
  -p voxel_size:=0.0
```

검출값 3회 확인

```
for i in {1..3}; do
  ros2 topic echo --once /sensor1/target_point
done
```

```
for i in {1..3}; do
  ros2 topic echo --once /sensor2/target_point
done
```

5B. RViz 단일 클릭 방식

```
source ~/drone_ws/install/setup.bash
rviz2 -d "$(ros2 pkg prefix grid_sensor_calibration)/share/grid_sensor_calibration/rviz/target_pick.rviz"
```

```
ros2 topic echo --once /clicked_point
```

6A. XY/yaw 노드 — 자동 원통 입력

터미널 5 — Sensor 1

```
source ~/drone_ws/install/setup.bash
CAL_SESSION="$(cat /tmp/grid_calibration_session)"
ros2 run grid_sensor_calibration sensor_calibration_node --ros-args \
  -r __node:=sensor1_calibration \
  -p sensor_name:=sensor1 \
  -p child_frame:=lidar1 \
  -p input_topic:=/sensor1/target_point \
  -p input_message_type:=point_stamped \
  -p capture_sample_count:=10 \
  -p auto_calibrate:=false \
  -p 'grid_points:=[0.0,0.0,12.0,0.0,24.0,0.0,0.0,6.0,12.0,6.0,24.0,6.0,0.0,15.0,12.0,15.0,24.0,15.0]' \
  -p calibration_file:="$HOME/grid_calibration/$CAL_SESSION/sensors_calibration.yaml" \
  -p nominal_x:=12.0 \
  -p nominal_y:=-2.0 \
  -p nominal_yaw:=-1.570796 \
  -p apply_tilt_correction:=true \
  -p sensor_z:=1.982 \
  -p sensor_roll:=-0.002025 \
  -p sensor_pitch:=0.000465
```

터미널 6 — Sensor 2

```
source ~/drone_ws/install/setup.bash
CAL_SESSION="$(cat /tmp/grid_calibration_session)"
ros2 run grid_sensor_calibration sensor_calibration_node --ros-args \
  -r __node:=sensor2_calibration \
  -p sensor_name:=sensor2 \
  -p child_frame:=lidar2 \
  -p input_topic:=/sensor2/target_point \
  -p input_message_type:=point_stamped \
  -p capture_sample_count:=10 \
  -p auto_calibrate:=false \
  -p 'grid_points:=[0.0,0.0,12.0,0.0,24.0,0.0,0.0,6.0,12.0,6.0,24.0,6.0,0.0,15.0,12.0,15.0,24.0,15.0]' \
  -p calibration_file:="$HOME/grid_calibration/$CAL_SESSION/sensors_calibration.yaml" \
  -p nominal_x:=12.0 \
  -p nominal_y:=17.0 \
  -p nominal_yaw:=1.570796 \
  -p apply_tilt_correction:=true \
  -p sensor_z:=1.979 \
  -p sensor_roll:=0.000279 \
  -p sensor_pitch:=-0.001633
```

6B. XY/yaw 노드 — `/clicked_point` 입력

터미널 3 — Sensor 1

```
source ~/drone_ws/install/setup.bash
CAL_SESSION="$(cat /tmp/grid_calibration_session)"
ros2 run grid_sensor_calibration sensor_calibration_node --ros-args \
  -r __node:=sensor1_calibration \
  -p sensor_name:=sensor1 \
  -p child_frame:=lidar1 \
  -p input_topic:=/clicked_point \
  -p input_message_type:=point_stamped \
  -p capture_sample_count:=1 \
  -p auto_calibrate:=false \
  -p 'grid_points:=[0.0,0.0,12.0,0.0,24.0,0.0,0.0,6.0,12.0,6.0,24.0,6.0,0.0,15.0,12.0,15.0,24.0,15.0]' \
  -p calibration_file:="$HOME/grid_calibration/$CAL_SESSION/sensors_calibration.yaml" \
  -p nominal_x:=12.0 \
  -p nominal_y:=-2.0 \
  -p nominal_yaw:=-1.570796 \
  -p apply_tilt_correction:=true \
  -p sensor_z:=1.982 \
  -p sensor_roll:=-0.002025 \
  -p sensor_pitch:=0.000465
```

터미널 4 — Sensor 2

```
source ~/drone_ws/install/setup.bash
CAL_SESSION="$(cat /tmp/grid_calibration_session)"
ros2 run grid_sensor_calibration sensor_calibration_node --ros-args \
  -r __node:=sensor2_calibration \
  -p sensor_name:=sensor2 \
  -p child_frame:=lidar2 \
  -p input_topic:=/clicked_point \
  -p input_message_type:=point_stamped \
  -p capture_sample_count:=1 \
  -p auto_calibrate:=false \
  -p 'grid_points:=[0.0,0.0,12.0,0.0,24.0,0.0,0.0,6.0,12.0,6.0,24.0,6.0,0.0,15.0,12.0,15.0,24.0,15.0]' \
  -p calibration_file:="$HOME/grid_calibration/$CAL_SESSION/sensors_calibration.yaml" \
  -p nominal_x:=12.0 \
  -p nominal_y:=17.0 \
  -p nominal_yaw:=1.570796 \
  -p apply_tilt_correction:=true \
  -p sensor_z:=1.979 \
  -p sensor_roll:=0.000279 \
  -p sensor_pitch:=-0.001633
```

7. 9점 캡처

```
0: ( 0,  0)
1: (12,  0)
2: (24,  0)
3: ( 0,  6)
4: (12,  6)
5: (24,  6)
6: ( 0, 15)
7: (12, 15)
8: (24, 15)
```

LiDAR 센서 좌표계 예상 위치

index	Grid (x, y) m	LiDAR 1 예상 (x, y) m	LiDAR 2 예상 (x, y) m
0	(0, 0)	(-2.00, -12.00)	(-17.00, +12.00)
1	(12, 0)	(-2.00, 0.00)	(-17.00, 0.00)
2	(24, 0)	(-2.00, +12.00)	(-17.00, -12.00)
3	(0, 6)	(-8.00, -12.00)	(-11.00, +12.00)
4	(12, 6)	(-8.00, 0.00)	(-11.00, 0.00)
5	(24, 6)	(-8.00, +12.00)	(-11.00, -12.00)
6	(0, 15)	(-17.00, -12.00)	(-2.00, +12.00)
7	(12, 15)	(-17.00, 0.00)	(-2.00, 0.00)
8	(24, 15)	(-17.00, +12.00)	(-2.00, -12.00)

```
LiDAR 1 automatic target point.z 예상 범위: -1.68 ~ -0.28 m
LiDAR 2 automatic target point.z 예상 범위: -1.68 ~ -0.28 m
허용 확인 범위: 예상 XY에서 약 1.0 ~ 1.5 m 이내
```

```
ros2 service call /sensor1_calibration/capture_point std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/capture_point std_srvs/srv/Trigger
```

```
ros2 service call /sensor1_calibration/status std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/status std_srvs/srv/Trigger
```

직전 캡처 취소

```
ros2 service call /sensor1_calibration/undo_point std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/undo_point std_srvs/srv/Trigger
```

특정 인덱스 재캡처

```
ros2 param set /sensor1_calibration capture_index 4
ros2 service call /sensor1_calibration/capture_point std_srvs/srv/Trigger
```

```
ros2 param set /sensor2_calibration capture_index 4
ros2 service call /sensor2_calibration/capture_point std_srvs/srv/Trigger
```

전체 초기화

```
ros2 service call /sensor1_calibration/reset std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/reset std_srvs/srv/Trigger
```

초기화 복구

```
ros2 service call /sensor1_calibration/restore std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/restore std_srvs/srv/Trigger
```

8. XY/yaw 계산

```
ros2 service call /sensor1_calibration/run_calibration std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/run_calibration std_srvs/srv/Trigger
```

```
ros2 service call /sensor1_calibration/status std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/status std_srvs/srv/Trigger
```

9. 운영 YAML 수정 및 빌드

```
nano ~/drone_ws/src/drone_localization/params/lidar_localization_os1.yaml
```

```
cd ~/drone_ws
source /opt/ros/humble/setup.bash
colcon build --packages-select drone_localization --symlink-install
source ~/drone_ws/install/setup.bash
```

10. 전체 PointCloud 겹침 확인

터미널 1 — PTP

```
sudo ptp4l -f ~/drone_ws/ptp_gm.conf -m
```

터미널 2 — 드라이버

```
source ~/drone_ws/install/setup.bash
ros2 launch dual_ouster_driver dual_os1.launch.py
```

터미널 3 — 전체 World Cloud 임시 발행

```
source ~/drone_ws/install/setup.bash
ros2 run drone_localization lidar_drone_tracker --ros-args \
  --params-file ~/drone_ws/src/drone_localization/params/lidar_localization_os1.yaml \
  -p publish_world_clouds:=true \
  -p publish_filtered_clouds:=false \
  -p debug_timing:=false
```

터미널 4 — RViz

```
source ~/drone_ws/install/setup.bash
rviz2 -d "$(ros2 pkg prefix dual_ouster_driver)/share/dual_ouster_driver/rviz/calibration_overlap.rviz"
```

11. 대회 운영

터미널 1 — PTP

```
cd ~/drone_ws
sudo ptp4l -f ~/drone_ws/ptp_gm.conf -m
```

터미널 2 — Dual LiDAR

```
source ~/drone_ws/install/setup.bash
ros2 launch dual_ouster_driver dual_os1.launch.py
```

터미널 3 — GUI

```
source ~/drone_ws/install/setup.bash
ros2 run competition_gui competition_gui
```

터미널 4 — 좌표 전용 RViz

```
source ~/drone_ws/install/setup.bash
ros2 run drone_localization estimate_viz --ros-args \
  -p show_position_marker:=false \
  -p show_text:=true \
  -p show_sigma_text:=false &

ESTIMATE_VIZ_PID=$!

rviz2 -d "$(ros2 pkg prefix dual_ouster_driver)/share/dual_ouster_driver/rviz/tracker_coordinates_only.rviz"

kill -INT "$ESTIMATE_VIZ_PID" 2>/dev/null
wait "$ESTIMATE_VIZ_PID" 2>/dev/null
```

12. 운영 확인

```
ros2 node list
ros2 topic hz /lidar1/points
ros2 topic hz /lidar2/points
ros2 topic hz /drone/estimated_pose --window 100
ros2 topic echo --once /drone/estimated_pose
```

```
ros2 param get /lidar_drone_tracker publish_world_clouds
ros2 param get /lidar_drone_tracker publish_filtered_clouds
ros2 param get /lidar_drone_tracker debug_timing
```

13. 처리 주기 디버깅

```
mkdir -p ~/competition_results
ip -s link show enp12s0
ros2 topic bw /lidar1/points
ros2 topic bw /lidar2/points
```

Cloud ON

```
source ~/drone_ws/install/setup.bash
ros2 run drone_localization lidar_drone_tracker --ros-args \
  --params-file ~/drone_ws/src/drone_localization/params/lidar_localization_os1.yaml \
  -p debug_timing:=true \
  -p debug_timing_period:=2.0 \
  -p publish_world_clouds:=true \
  -p publish_filtered_clouds:=true \
  2>&1 | tee ~/competition_results/debug_timing_cloud_on.log
```

Cloud OFF

```
source ~/drone_ws/install/setup.bash
ros2 run drone_localization lidar_drone_tracker --ros-args \
  --params-file ~/drone_ws/src/drone_localization/params/lidar_localization_os1.yaml \
  -p debug_timing:=true \
  -p debug_timing_period:=2.0 \
  -p publish_world_clouds:=false \
  -p publish_filtered_clouds:=false \
  2>&1 | tee ~/competition_results/debug_timing_cloud_off.log
```

14. 종료

```
pkill -INT -f "ground_plane_node|pointcloud_target_node|sensor_calibration_node"
pkill -INT -f "sensor_tf_publisher|point_transform_node|calibration_viz_node"
pkill -INT -f "lidar_localization.launch.py|lidar_drone_tracker"
pkill -INT -f "competition_gui|compet_score|estimate_viz"
pkill -INT -x rviz2
pkill -INT -f "dual_os1.launch.py"
pkill -INT -x os_driver
sudo pkill -INT -x ptp4l
```

```
ros2 node list
```